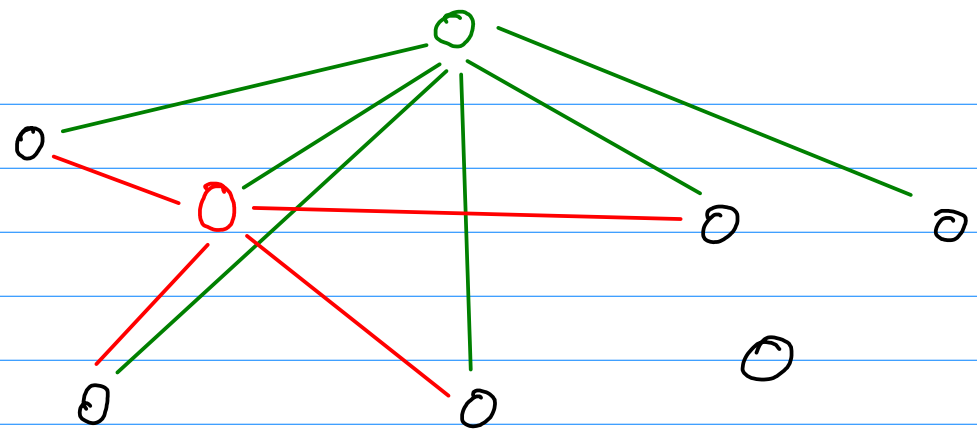
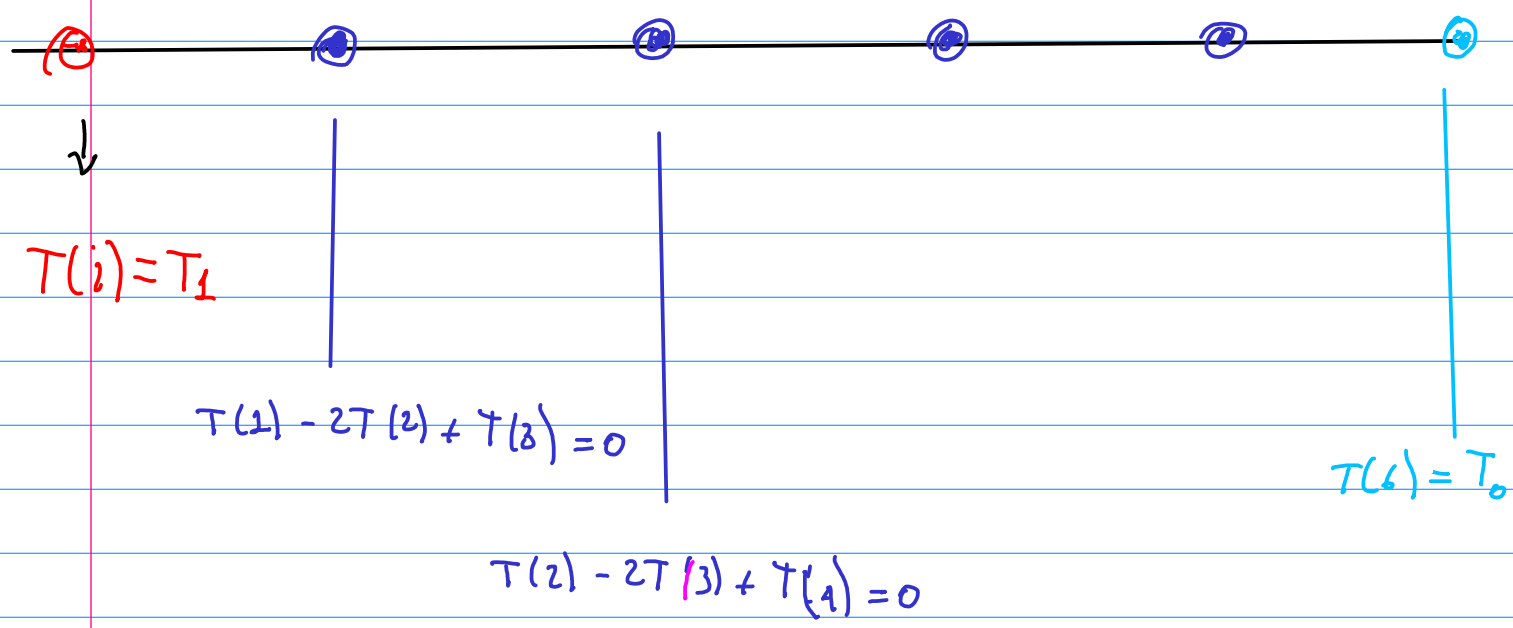


for partícula en partículas
  for partícula en vecinos (partícula)
    - calcula la distancia
    - calcula la fuerza

$$\frac{d^2T}{dx^2} = \frac{dT}{d\tilde{t}}$$

$$\frac{dT}{d\tilde{t}} \to 0$$

$$\frac{d^2}{dx^2} \to \frac{T(i+1) - 2T(i) + T(i-1)}{\Delta x^2} = 0$$



¿de qué tamaño es la matriz que tengo que resolver en este ejemplo?

Tendremos un sistema de 6 x 6 aunque dos ecuaciones son condiciones de frontera

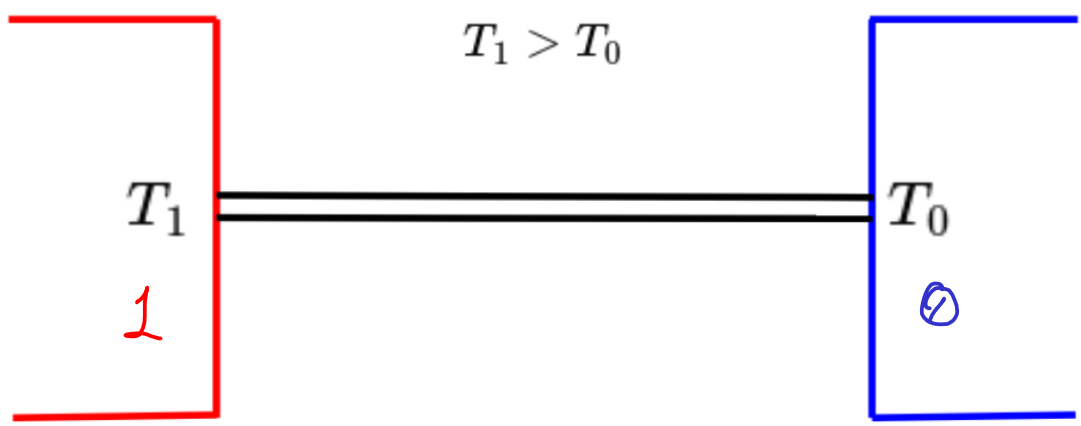
$$\begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & -2 & 1 & 0 & 0 & 0 \\ 0 & 1 & -2 & 1 & 0 & 0 \\ 0 & 0 & 1 & -2 & 1 & 0 \\ 0 & 0 & 0 & 1 & -2 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} T_1 \\ T_2 \\ T_3 \\ T_4 \\ T_5 \\ T_6 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

1

Queremos resolver numéricamente

$$\frac{d^2T}{dx^2} = 0$$

$$T_1 > T_0$$



Sabemos que tenemos que invertir una matriz A como la 1. Para esto necesitamos

Problema de entrada y salida de datos.

* Un arreglo 1D para la malla de tamaño n.
* Un arreglo bidimensional para guardar los valores de la matriz A (nxn)

* Un vector para las incógnitas (n)
* Un vector de condiciones de frontera (2)

* Una función que permita invertir la matriz. Podría requerir otra matriz.

Algoritmo a seguir

1) Inicializar las variables (definición de variables y asignación de valores iniciales)

2) Ensamblar la matriz

   Recorrer los renglones de la matriz

   asignando valores a cada entrada

3) Invertir la matriz.

$$\begin{pmatrix} b_1 & c_1 & & & & \\ a_2 & b_2 & c_2 & & & \\ 0 & a & b & c & & \\ 0 & 0 & a & b & c & \\ & & & & \ddots & \\ & & & & a_n & b_n \end{pmatrix} \begin{pmatrix} xx(1) \\ \vdots \\ xx(nn) \end{pmatrix} = \begin{pmatrix} rr(1) \\ \vdots \\ rr(nn) \end{pmatrix}$$

$b_1 xx(1) + c_1 xx(2) = rr(1)$

$$\begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & -2 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & -2 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & -2 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & -2 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & -2 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix}$$

matrices 'sparse'

Row major C

$$\begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix} \begin{pmatrix} 1 & -2 & 1 & 0 & 0 & 0 & 0 \end{pmatrix}$$

...

Column major

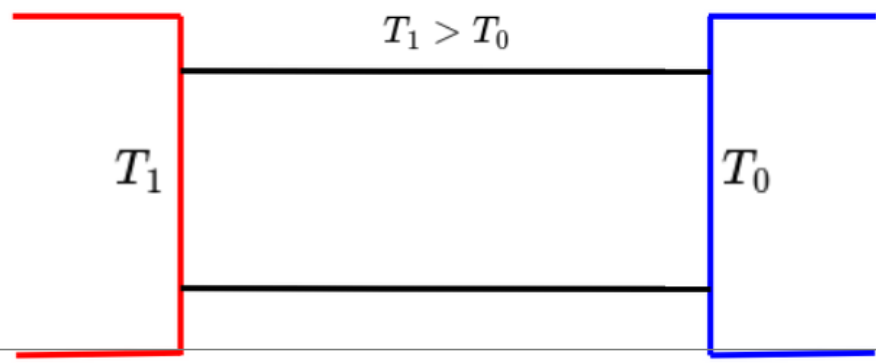
Fortran

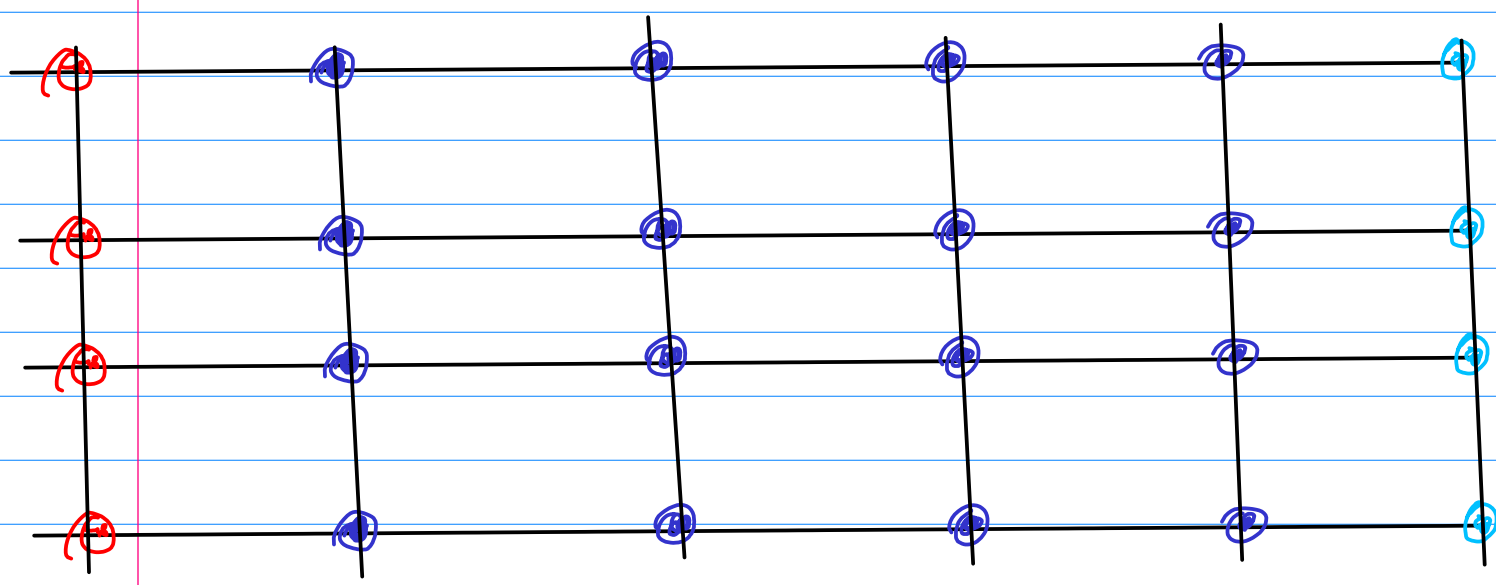
$$\begin{matrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ -2 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \end{matrix}$$

Problema bidimensional

Resolver la ecuación de Laplace en una placa 2D, de longitud L

$$\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} = 0$$





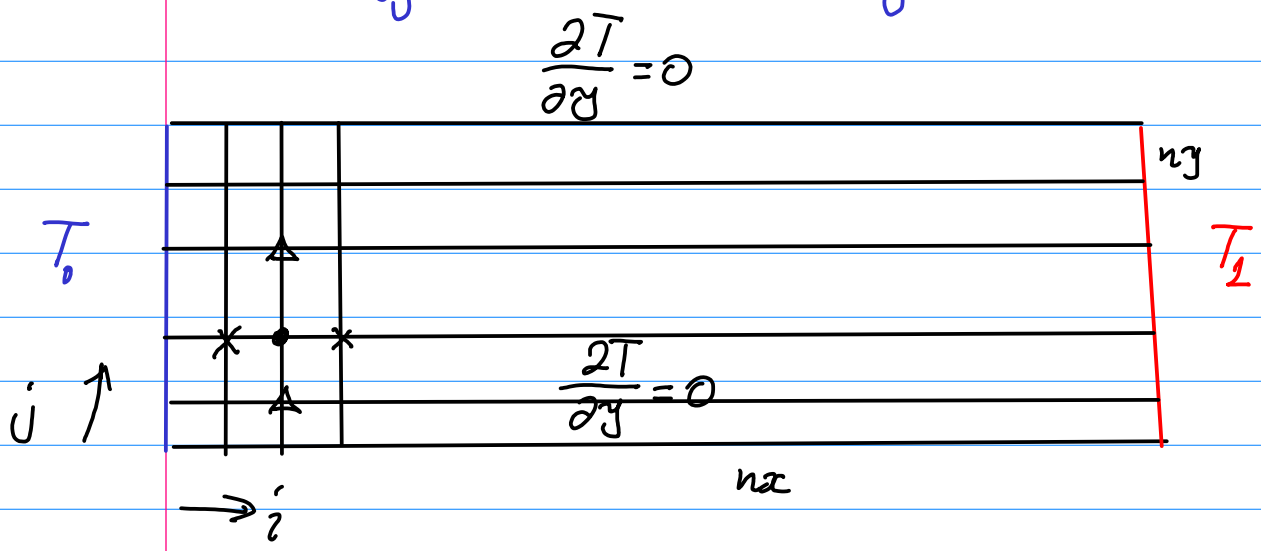
¿Podemos aprovechar el programa de la tridiagonal que usamos en 1D?

No directamente, necesitamos un proceso iterativo.

$$\frac{T(x_0-h_x,y_0)+T(x_0+h_x,y_0)}{h_x^2}+\frac{T(x_0,y_0-h_y)+T(x_0,y_0+h_y)}{h_y^2}-T(x_0,y_0)\left(\frac{2}{h_x^2}+\frac{2}{h_y^2}\right)=0$$

$$\Rightarrow \frac{1}{h_x^2}T(x_0-h_x,y_0)-\left(\frac{2}{h_x^2}+\frac{2}{h_y^2}\right)T(x_0,y_0)+\frac{1}{h_x^2}T(x_0+h_x,y_0)$$

$$=-\frac{1}{h_y^2}T(x_0,y_0-h_y)-\frac{1}{h_y^2}T(x,y_0+h_y)$$



$$\nabla^2 T=0$$

Estructuras de datos

coordenadas: $xx(:)$, $yy(:)$

inversor: $aa(:)$, $bb(:)$, $cc(:)$, $rr(:)$

resultados: $tt(:,:)$

cond_front: $cfx(ny,2)$, $cfy(nx,2)$

Para x usamos dimensión nx y para y ny.

Algoritmo

Estado inicial

Bucle de iteraciones

Bucle de barrido de y con ny pasos

Bucle de llenado de la matriz tridiagonal
aa, bb, cc, rr,

Impone cond. front.

Resuelve la línea en dirección x

fin del Bucle de y

Bucle de barrido de x con nx pasos

Bucle de llenado de la matriz tridiagonal
aa, bb, cc, rr,

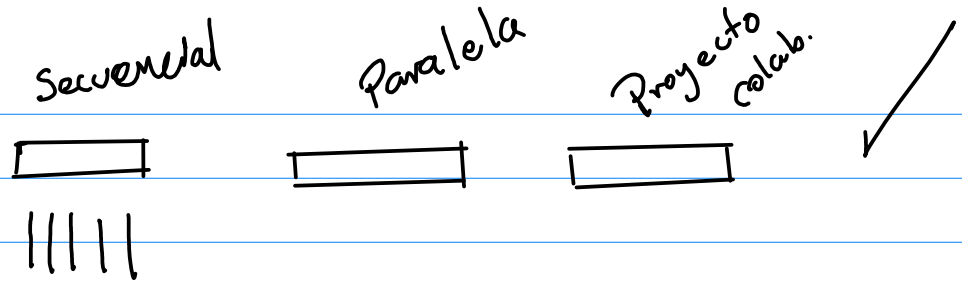
Impone cond. front.

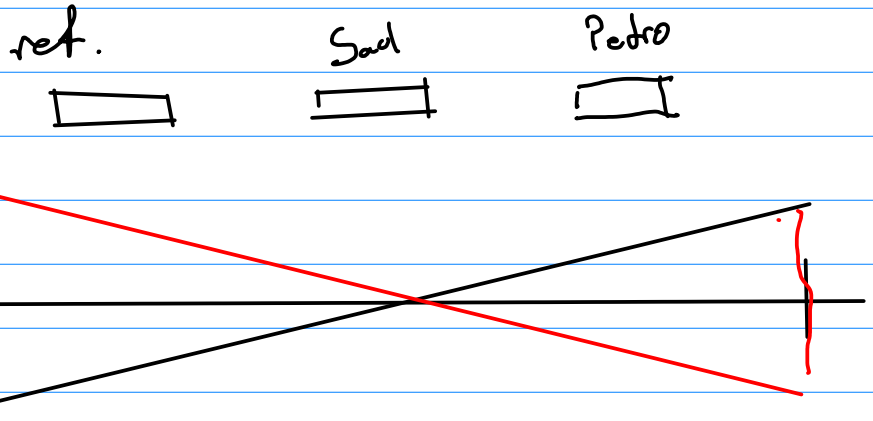
Resuelve la línea

fin del Bucle de x

Calcular la norma del residuo $\|T - T^*\|$ (T^*: T anterior)

fin de Bucle de iteraciones

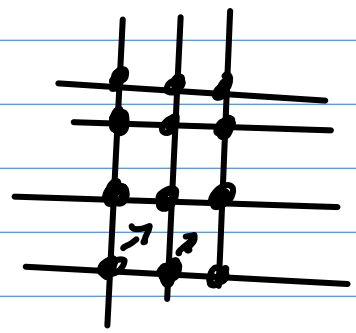




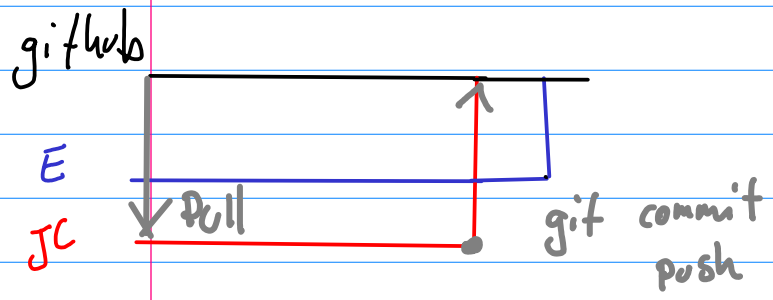
a_1	a_2	a_3	a_4	a_5
b_1	b_2	b_3	b_4	b_5

maxVal( )

$\Delta x \Delta y \, (T_{new} - T_{old})$



main




```
2027  git status
2028  git diff
2029  git status
2030  git add calor2D.f90
2031  git commit -m"prueba de conflicto 2.1"
2032  git push
2033  git status
2034  git pull
2035  git reset HEAD~1
2036  git status
2037  git diff
2038  git pull
2039  git stash save
2040  git stash list
2041  git status
2042  git diff
2043  git pull
2044  rm calor2D
2045  git pull
2046  ls
2047  ls -lrt
2048  git stash pop
2049  git status
2050  git commit -m"Fin prueba conflicto"
2051  git push
2052  history
```

deshace el commit

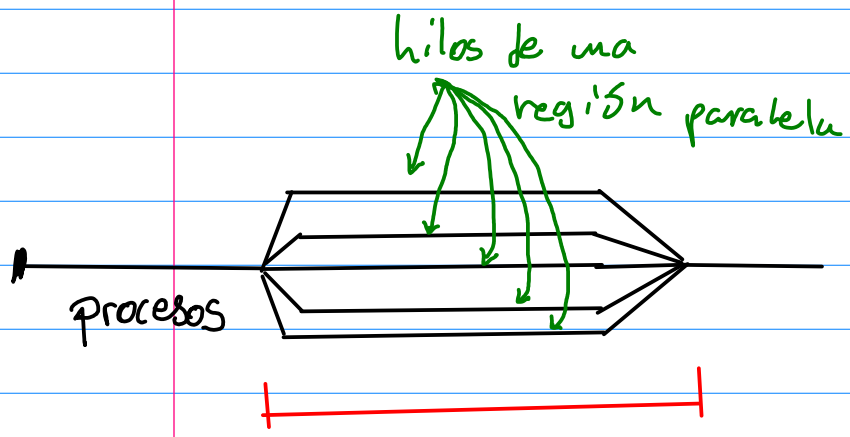
guarda modificaciones en un área sin seguimiento

actualizamos con el repositorio

recupera nuestros cambios sin seguimiento

Introducción OpenMP.

* Estándar de paralelización basado en directivas
* Ofrece buena portabilidad
* Tiene más de 20 años (CPUs)
* Paralelismo de memoria compartida (a nivel de nodo de cómputo)



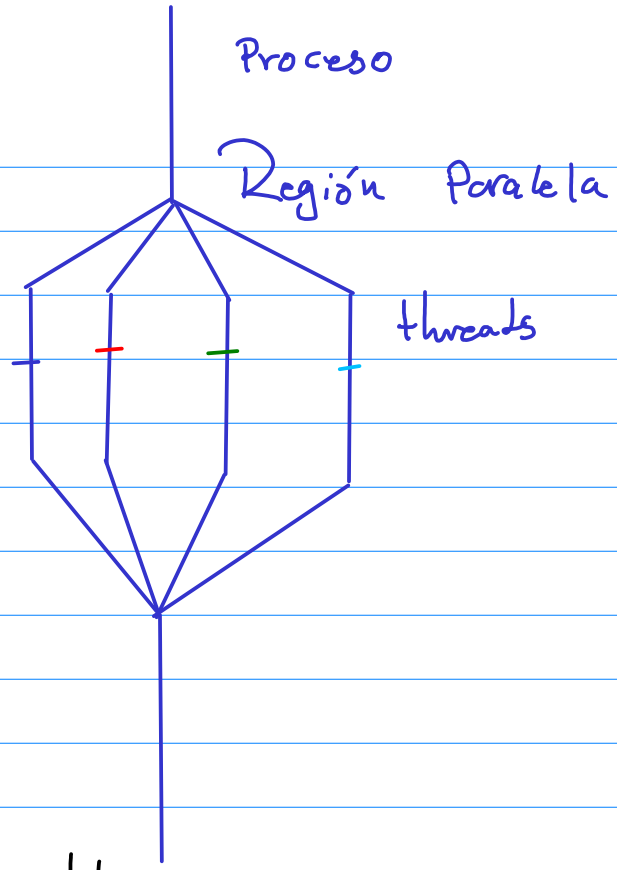
Región paralela con n hilos 'threads'

Para diferentes lenguajes de programación tenemos distintos sentencias

```
!$ omp parallel   clausula 1   clausula 2 ...
...
!$ OMP END Parallel
```

OpenMP es una colección de directivas y de funciones intrínsecas.

- → compartir trabajo
- → sincronización
- → Manejo de datos



Se ejecuta el mismo código en todos los hilos

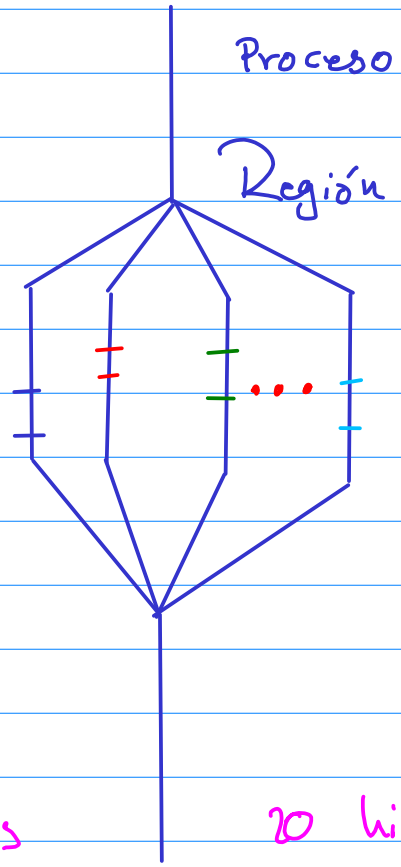
¡¡ojo!! el resultado de los programas paralelos que usan openmp puede variar entre ejecuciones dependiendo del orden en el que los hilos realizan trabajo.

* Existe un comportamiento por defecto para cada tipo de estructura de datos.

* Hay dos tipos principales de variables en openmp.

a) Compartida. Es una variable donde todos los hilos de un proceso pueden leer y escribir

b) Privada. Es una variable para cada hilo donde el único que puede escribir es el hilo propietario.



20 hilos 20 hilos

núcleos de procesamiento.

¿Cómo podemos sumar los primeros N números naturales usando solo regiones paralelas?

Podemos usar los identificadores de hilo privados omp_get_thread_num() para sumarlos en una variable compartida.

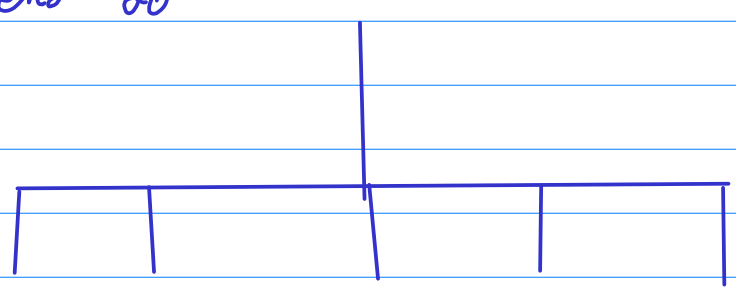
Operator/Intrinsic	Initialization
+	0
*	1
-	0
.AND.	.TRUE.
.OR.	.FALSE.
.EQV.	.TRUE.
.NEQV.	.FALSE.
MAX	smallest representable number
MIN	largest representable number
IAND	all bits on
IOR	0
IEOR	0

Table 3.1: Initialization rules for variables included in `REDUCTION` clauses.

Existe otro constructor para compartir trabajo

```
!$OMP DO clausula 1 clausula 2 ...
  ...
!$omp end do
```

!$omp do



i = 1, 10 i = 11, 20 i = 21, 30 ...

!$omp end do

Se destruyen las copias de variables privadas y se actualizarán los valores de las variables compartidas.

Los índices del bucle son privados por defecto, pero no lo declaramos explícitamente.

Programar un producto matriz-vector con regiones paralelas y parallel do.

$A\ (n \times n)$ $\vec{v}\ (n \times 1)$

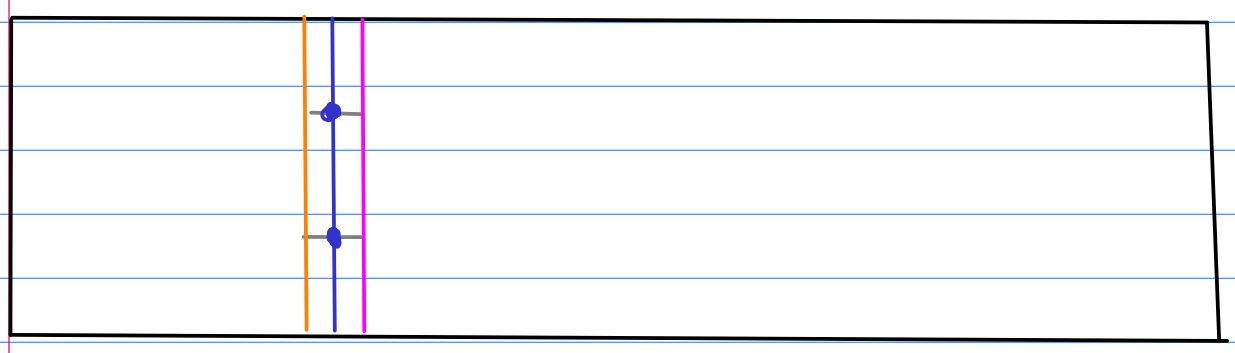
Para resolver este problema necesitamos el arreglo de la matriz A y del vector.

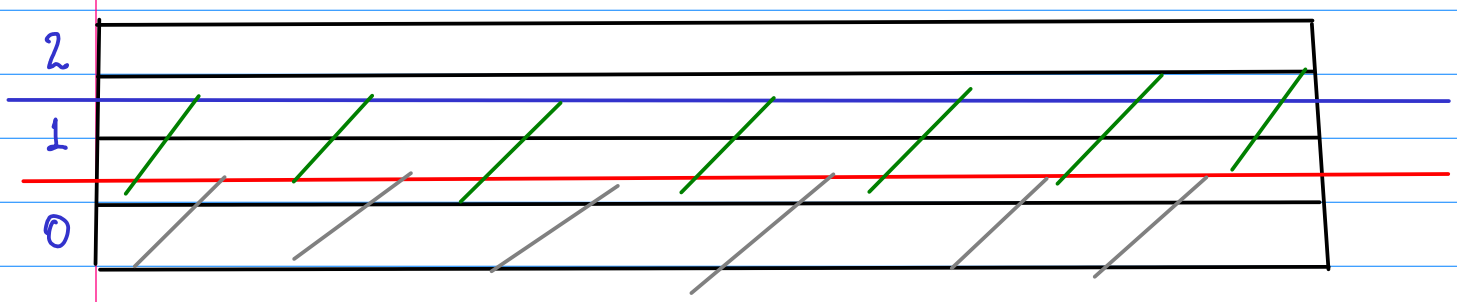
En la región paralela pueden ser compartidos porque nadie escribe en ellos.

* Recomendación: Todas las variables que sean exclusivamente de lectura en una región paralela pueden ser compartidas

Si hay escritura dependiente de índices no compartidos entre hilos, la variable puede ser compartida.

Analizar la estructura del programa calor1D.490 de la rama referencia para identificar regiones paralelizables y tipos de variables.




2
1
0

Misión 19 marzo

* Paralelizar el bucle ensambla_tri_x de calor2D.f90

1) Lista de variables públicas
2) Lista de variables privadas

En este caso todas las variables pueden ser compartidas, en especial jj debe ser compartida.

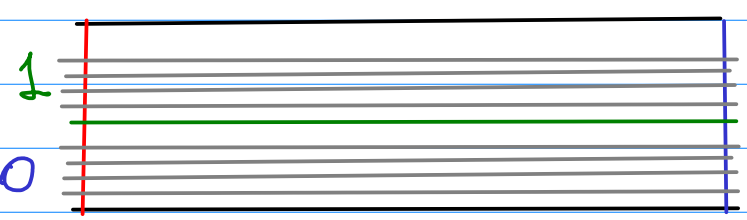
Vemos que el tiempo de ejecución del programa paralelizado aumenta. Esto es porque crear los hilos cuesta tiempo, y si el trabajo que hace cada uno es poco no acelera la ejecución del programa.

* * Analizar el bucle barrido_y.

¿es paralelizable?

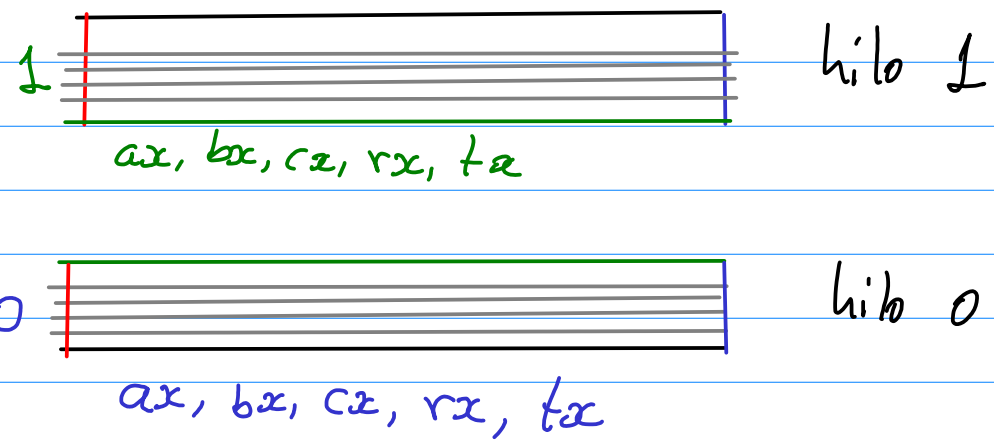
En caso de que sí,

a) dar una lista de variables públicas
b) dar una lista de variables privadas.



construye A (tridiagonal) y la invierte

Respuesta: Dada la naturaleza iterativa del algoritmo de solución que tenemos es posible paralelizar el bucle sin peligro de obtener resultados erróneos.



Se resuelve de manera secuencial en cada hilo, partiendo del resultado de la iteración anterior.

Cada hilo va ensamblando y resolviendo los sistemas tridiagonales de manera paralela, por lo que necesitamos que los arreglos del problema algebraico sean privados.

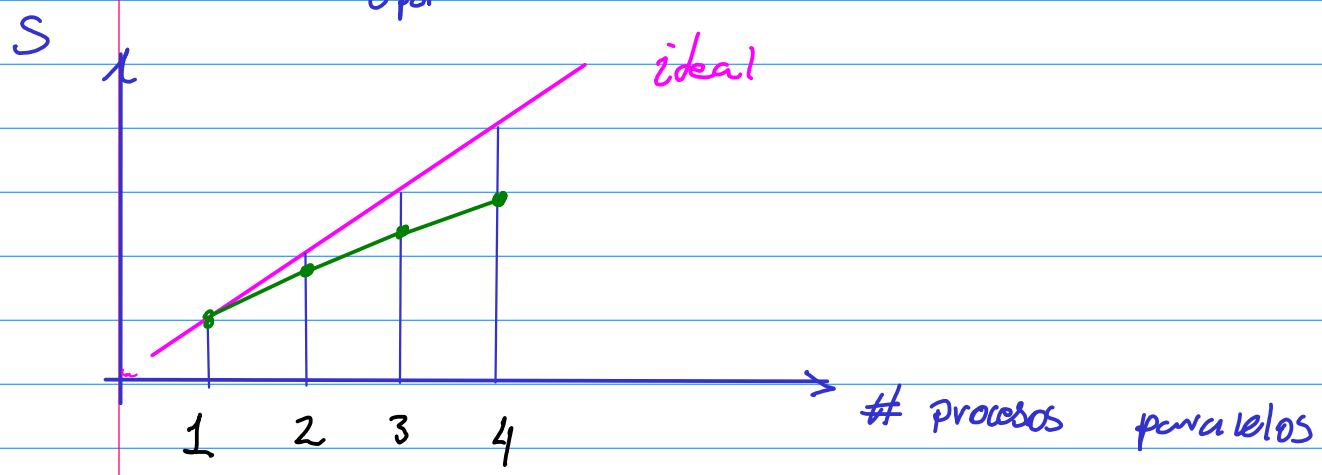
Notamos que la cantidad de operaciones que se asigna a cada hilo, es mucho mayor que la que habíamos logrado paralelizando el bucle ensambla_fri_x. Tenemos un paralelismo de

grano más grueso.

Evaluación de la aceleración paralela

* "speed up": es el cociente entre el tiempo de una ejecución secuencial y una ejecución paralela con p procesadores

$$S = \frac{t_{sec}}{t_{par}}$$



Actividad: 1) Construir un caso de prueba de transferencia de calor 2D de 128 x 128 puntos

2) Calcular t_s usando el comando time

3) usar la variable de entorno

OMP_NUM_THREADS para ejecutar el mismo caso usando 2 procesadores, 3, etc...

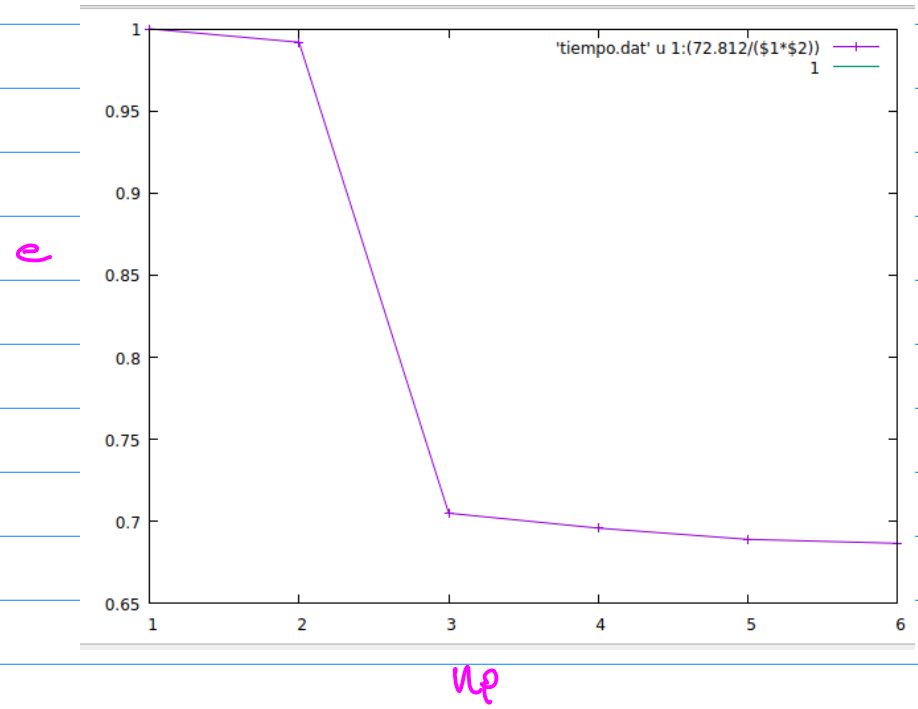
b) Hacer una gráfica del speed up que se obtiene

Otra métrica que expresa la calidad de la paralelización de un programa es la eficiencia paralela

$$e = \frac{t_{sec}}{n_p * t_p}$$

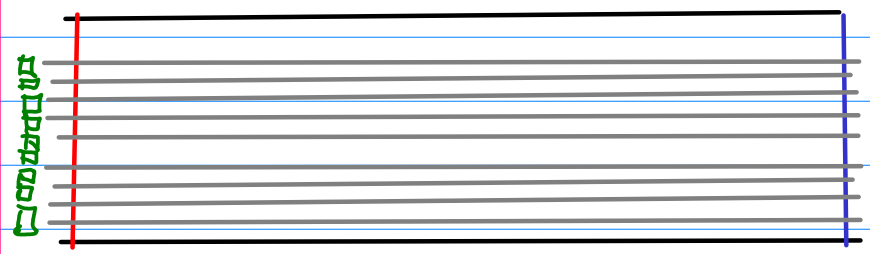
; t_{sec}: tiempo de la ejecución secuencial

t_p: tiempo con n_p procesos paralelos



Rediseño del programa calor2D.4go para paralelismo de grano más fino.
Restringidos al uso del inversor tridiagonal.

En lugar de separar con grupos de líneas, separamos líneas individuales.



Esta idea es muy atractiva pero plantea preguntas importantes

* ¿cómo pasamos a un dominio de dirección con líneas individuales?

Tenemos que modificar la estructura de los datos que definen los problemas tridiagonales.

* ¿Qué ocurre con la memoria que se necesitaría? Aumenta significativamente porque queremos evitar condiciones de carrera. Hay que hacer la cuenta.